{EPITECH}

DAY 06_

< POINTERS ARE BACK />

DAY 06

Preliminaries

Language: C

The totality of your source files, except all useless files (binary, temp files, obj files,...), must be included in your delivery.

- ✓ Don't push your `main` function into your delivery directory, we will be adding our own. Your files will be compiled adding our `main.c` and our `my_putchar.c` files.
- ✓ You are only allowed to use the `my_putchar` function to complete the following tasks, but **don't push it** into your delivery directory, and don't copy it in *any* of your delivered files.
- ✓ If one of your files prevents you from compiling with `*.c`, the Autograder will not be able to correct your work and you will receive a 0.

Clone your repository at the beginning of the day and submit your work on a regular basis!
The delivery directory is specified within the instructions for each task.
In order to keep your repository clean, pay attention to `gitignore`.

Most of the day's functions exist in the **string** library. Use `man` to obtain a full explanation of how a function works. Beware that none of your deliveries contains a function from this **string** library!

{EPITECH}

Introduction - Unit Tests

It is highly recommended to test your functions as you develop them. It is common practice to create and use what is called **unit tests**.

To help you write good unit tests in C, you will use the Criterion framework. To do that, please follow the instructions on the document "How to write Unit Tests" from the intranet, available here.

> You don't know Makefile yet, so your test will be built manually using the command shown in the documentation.

Task 01 - my_strcpy

Delivery: my_strcpy.c

Write a function that copies a string into another. The destination string will already have enough memory to copy the source string.
It must be prototyped the following way:

```
char *my_strcpy(char *dest, char const *src)
```

The function returns `dest`.

Task 02 - my_strncpy

Delivery: my_strncpy.c

Write a function that copies `n` characters from a string into another.
The destination string will already have enough memory to contain `n` characters.
It must be prototyped the following way:

```
char *my_strncpy(char *dest, char const *src, int n);
```

The function returns `dest`.

Add '\0's if `n` is strictly greater than the length of the string. Do not add '\0' if `n` is strictly lower than the length of the string (because `dest` is not supposed to contain more than `n` bytes.)

Task 03 - my_revstr

Delivery: my_revstr.c

Write a function that reverses a string. It must be prototyped the following way:

```
char *my_revstr(char *str);
```

The function returns `str`.

Task 04 - Unit tests - part I

Delivery: tests/test_my_strncpy.c, tests/test_my_revstr.c

As ask in the introduction, you have to write unit tests (using Criterion) for some tasks.
Starting now with the units tests for the following functions: `my_strncpy` and `my_revstr`.
You **MUST** have at least a line coverage of 60% and a branch coverage of 40% for both of these functions.

Here's an example of unit test for the function `my_strncpy`:

```
#include <criterion/criterion.h>

Test(my_strncpy, copy_five_characters_in_empty_array)
{
    char dest[6] = {0};

    my_strncpy(dest, "HelloWorld", 5);
    cr_assert_str_eq(dest, "Hello");
}
```

You can even compare the result with the libC functions like so:

```
Test(my_strncpy, copy_string_in_empty_array)
{
        char my_dest[6] = {0};
        char dest[6] = {0};

        my_strncpy(my_dest, "Hello", 6);
        strncpy(dest, "Hello", 6);
        cr_assert_str_eq(my_dest, dest);
```

```
}
```

Task 05 - my_strstr

Delivery: my_strstr.c

Reproduce the behavior of the `strstr` function. Your function must be prototyped the following way:

```
char *my_strstr(char *str, char const *to_find);
```

> Check out the `my_strcmp` and `my_strncmp` functions.

Task 06 - my_strcmp

Delivery: my_strcmp.c

Reproduce the behavior of the `strcmp` function. Your function must be prototyped the following way:

```
int my_strcmp(char const *s1, char const *s2);
```

Task 07 - my_strncmp

Delivery: my_strncmp.c

Reproduce the behavior of the `strncmp` function. Your function must be prototyped the following way:

```
int my_strncmp(char const *s1, char const *s2, int n);
```

The function should return the same values as `man 3 strcmp`.

Task 08 - my_strupcase

Delivery: my_strupcase.c

Write a function that puts every letter of every word in it in uppercase.
It must be prototyped the following way:

```
char *my_strupcase(char *str);
```

The function returns `str`.

Task 09 - my_strlowcase

Delivery: my_strlowcase.c

Write a function that puts every letter of every word in it in lowercase.
It must be prototyped the following way:

```
char *my_strlowcase(char *str);
```

The function returns `str`.

Task 10 - my_strcapitalize

Delivery: my_strcapitalize.c

Write a function that capitalizes the first letter of each word.
It must be prototyped the following way:

```
char *my_strcapitalize(char *str);
```

The function returns `str`.

The sentence, `hey, how are you? 42WORds forty-two; fifty+one`
will become `Hey, How Are You? 42words Forty-Two; Fifty+One`

Task 11 - Unit tests - part II

Delivery: tests/test_my_strstr.c, tests/test_my_strncmp.c, tests/test_my_strcapitalize.c

You now have to write more unit tests (using Criterion) for the following functions: `my_strstr`, `my_strncmp` and `my_strcapitalize`.
You **MUST** have at least a line coverage of 80% and a branch coverage of 60% for all of these functions.

Task 12 - my_str_isalpha

Delivery: my_str_isalpha.c

Write a function that returns `1` if the string passed as parameter only contains alphabetical characters and `0` if the string contains another type of character.
It must be prototyped the following way:

```
int my_str_isalpha(char const *str);
```

The function returns `1` if `str` is an empty string.

Task 13 - my_str_isnum

Delivery: my_str_isnum.c

Write a function that returns `1` if the string passed as parameter only contains digits and `0` otherwise.
It must be prototyped the following way:

```
int my_str_isnum(char const *str);
```

The function returns `1` if `str` is an empty string.

Task 14 - my_str_islower

Delivery: my_str_islower.c

Write a function that returns `1` if the string passed as parameter only contains lowercase alphabetical characters and `0` otherwise.
It must be prototyped the following way:

```
int my_str_islower(char const *str);
```

The function returns `1` if `str` is an empty string.

Task 15 - my_str_isupper

Delivery: my_str_isupper.c

Write a function that returns `1` if the string passed as parameter only contains uppercase alphabetical characters and `0` otherwise.
It must be prototyped the following way:

```
int my_str_isupper(char const *str);
```

The function returns `1` if `str` is an empty string.

Task 16 - my_str_isprintable

Delivery: my_str_isprintable.c

Write a function that returns `1` if the string passed as parameter only contains printable characters and `0` otherwise.
It must be prototyped the following way:

```
int my_str_isprintable(char const *str);
```

The function returns `1` if `str` is an empty string.

man isprint

Task 17 - Unit tests - part III

Delivery: tests/test_my_str_isalpha.c, tests/test_my_str_islower.c, tests/test_my_str_isprintable.c

You now have to write more unit tests (using Criterion) for the following functions: `my_str_isalpha`, `my_str_islower` and `my_str_isprintable`.
You **MUST** have at least a line coverage of 100% and a branch coverage of 80% for all of these functions.

Task 18 - my_putnbr_base

Delivery: my_putnbr_base.c

Write a function that converts and displays a decimal number into a number in a given base.
The number is given as an `int` and the base is provided as a `string`.
The base contains all the symbols that can be used to print a number (for instance, `0123456789` for the decimal base, `01` for the binary base, `0123456789ABCDEF` for the hexadecimal base).
The function must deal with negative numbers, and be prototyped the following way:

```
int my_putnbr_base(int nbr, char const *base);
```

Task 19 - my_getnbr_base

Delivery: my_getnbr_base.c

Write a function that converts and returns a number (provided as a `string`) in a given base into a decimal number.
The function must deal with negative numbers, and several successive `+` or `-` before the number.
If any error occurs, the function must return `0`. It must be prototyped the following way:

```
int my_getnbr_base(char const *str, char const *base);
```

Task 20 - my_showstr

Delivery: my_showstr.c

Write a function that prints a string and returns `0`. If this string contains non-printable characters, they must be printed hexadecimally (in lowercase letters) with a backslash before the given value.
It must be prototyped the following way:

{EPITECH}

```
int my_showstr(char const *str);
```

For instance, `I like \n ponies!` will be printed as `I like \0a ponies!`.

Task 21 - my_showmem

Delivery: my_showmem.c

Write a function that prints a memory dump and return `0`. It must be prototyped the following way:

```
int my_showmem(char const *str, int size);
```

Each line of the output manages 16 characters and is divided into three columns:

- ✓ The hexadecimal address of the line's first character,
- ✓ the content in hexadecimal,
- ✓ the content in printable characters.

Any non printable characters must be replaced by a dot.

Terminal

```
$>  ./my_showmem | cat -e
00000000: 6865 7920 6775 7973 2073 686f 7720 6d65 hey guys show me$
00000010: 6d20 6973 2063 6f6f 6c20 796f 7520 6361 m is cool you ca$
00000020: 6e20 646f 2073 6f6d 6520 7072 6574 7479 n do some pretty$
00000030: 206e 6561 7420 7374 7566 6600 0f1b 7f05  neat stuff.....$
00000040: 2e00 0102 0304 0506 0708 090e 0f1b 7f    ...............$
```

Don't forget the padding if there aren't enough characters to have a valid alignment.

v 2.1